Architecture des Ordinateurs : Représentation des Entiers et Arithmétique en Complément à Deux

Thème principal : Représentation binaire des entiers signés et arithmétique bas niveau

Synthèse exécutive : Ce cours aborde la manière dont les ordinateurs stockent et manipulent physiquement les entiers en mémoire. En l'absence de composants électroniques capables d'enregistrer directement un signe '+' ou '-', l'architecture informatique utilise la convention du complément à deux sur un nombre fixe de bits. Ce système unifie l'addition et la soustraction au sein de l'Unité Arithmétique et Logique (ALU) tout en éliminant le problème du double zéro présent dans la représentation 'Signe et Magnitude'. Sont détaillés la méthode de conversion en trois étapes, le calcul de la plage de valeurs dissymétrique, la gestion de la retenue sortante (Carry out) et la règle stricte de détection du dépassement de capacité (Overflow).

Problématique matérielle et justification du Complément à Deux

Au niveau le plus bas de l'architecture matérielle, la mémoire et les registres processeur sont formés de circuits électroniques bistables (bascules). Ces composants ne reconnaissent que deux niveaux de tension : l'absence de tension (0 Volt, 0 logique) et la présence de tension (Vcc, 1 logique). Il est physiquement impossible d'inscrire directement un glyphe comme '+' ou '-' dans un transistor. Pour représenter des entiers relatifs, les concepteurs ont d'abord envisagé le codage 'Signe + Magnitude' (où le bit le plus à gauche indique le signe et les autres la valeur absolue). Cependant, ce codage génère deux représentations pour zéro (+0 = 0000_2 et -0 = 1000_2 sur 4 bits), ce qui gaspille une combinaison binaire, complexifie les tests d'égalité et impose d'ajouter un circuit de soustraction distinct dans l'Unité Arithmétique et Logique (ALU). Le complément à deux résout totalement ces problèmes : zéro possède un codage unique (0000...0) et la soustraction A - B s'effectue exactement comme l'addition A + (-B) en réutilisant le même additionneur binaire.

- Absence de signe physique : les transistors et bascules bistables ne stockent que deux états (0 Volt ou Vcc)
- Limites du codage 'Signe et Magnitude' : existence d'un double zéro (+0 et -0) et nécessité de circuits séparés dans l'ALU
- Unification de l'ALU : la soustraction A - B est ramenée à une addition A + (-B) sur le même circuit

Cas concret enseignant : *Le professeur utilise la métaphore du compteur kilométrique d'une voiture ancienne à 4 chiffres : si le compteur affiche 0000 km et que vous faites marche arrière d'un kilomètre, le compteur repasse mécaniquement à 9999. En binaire sur 8 bits, faire 00000000_2 - 1 donne 11111111_2, ce qui définit formellement la représentation de -1 en complément à deux.*

Algorithme de Conversion et Plage de Valeurs sur N bits

Pour convertir un nombre négatif -X en complément à deux sur un mot mémoire de taille fixe N bits, il faut suivre une procédure séquentielle rigoureuse en trois étapes. Premièrement, on écrit la valeur absolue X en binaire pur et on la cadre impérativement sur N bits en complétant par des zéros à gauche. Deuxièmement, on applique le complément à un en inversant chaque bit (les 0 deviennent 1, les 1 deviennent 0). Troisièmement, on ajoute 1 au résultat binaire obtenu. Le bit le plus à gauche (MSB) prend la valeur 1 pour tout nombre négatif et 0 pour tout nombre positif ou nul. La plage des nombres représentables n'est pas symétrique : sur 8 bits, elle s'étend de -128 à +127. Cette dissymétrie provient du fait que le zéro (00000000) possède un MSB égal à 0 et est rattaché au domaine positif.

- Méthode des 3 étapes : Valeur absolue sur N bits, Inversion des bits (Complément à 1), Ajout de 1 (Complément à 2)
- Rôle du MSB (Most Significant Bit) : bit de poids fort indiquant le signe (0 pour positif ou nul, 1 pour négatif)
- Dissymétrie de la plage : de -2^(N-1) à +(2^(N-1) - 1) en raison de l'inclusion du zéro dans les positifs

Cas concret enseignant : *Avertissement crucial du professeur pour les partiels : il est IMPÉRATIF de fixer la taille N bits (ex: 8 bits) et d'ajouter les zéros de gauche AVANT d'effectuer l'inversion des bits. Inverser la séquence '1100' (+12 sur 4 bits) sans écrire les zéros de cadrage '00001100' donne '0011' au lieu de '11110011', ce qui conduit à une réponse totalement fausse en examen !*

Arithmétique, Retenue (Carry Out) et Détection du Dépassement de Capacité (Overflow)

En travaillant sur des registres de largeur fixe (8, 16, 32 ou 64 bits), les opérations arithmétiques peuvent générer deux drapeaux distincts au niveau du processeur : le Carry Out et l'Overflow (OV). Le Carry Out représente la retenue sortante du bit MSB ; en arithmétique signée en complément à deux, cette retenue est rejetée car l'architecture effectue ses calculs modulo 2^N. L'Overflow, en revanche, indique une erreur d'amplitude où le résultat mathématique réel excède la plage représentable [-2^(N-1), +2^(N-1)-1]. La règle formelle de détection d'un Overflow en examen est stricte : un Overflow se produit si et seulement si l'on additionne deux nombres de même signe (deux positifs ou deux négatifs) et que le résultat possède un signe inverse. Si l'on additionne deux nombres de signes différents, l'Overflow est physiquement et mathématiquement impossible.

- Soustraction unifiée : calculée sous la forme A + (-B) au sein de l'ALU
- Carry Out (retenue sortante) : retenue au-delà du MSB, systématiquement ignorée en arithmétique signée modulo 2^N
- Règle de l'Overflow : dépassement survenant si et seulement si deux opérandes de même signe additionnés produisent un résultat de signe opposé

Cas concret enseignant : *Le professeur cite l'anecdote historique du crash du vol 501 de la fusée Ariane 5 en 1996 : un nombre flottant sur 64 bits a été converti en entier signé sur 16 bits sans mécanisme de protection contre l'Overflow. Le dépassement de capacité non géré a corrompu les données du système de guidage, provoquant le braquage brutal des tuyères puis l'autodestruction de la fusée.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la retenue tout à gauche (Carry sortant) est-elle jetée lors de l'addition en complément à deux ?
Réponse du professeur : Parce que le processeur travaille sur des registres à taille fixe (modulo 2^N). Les calculs en complément à deux s'auto-régulent mathématiquement dans cet espace modulaire, ce qui rend la retenue sortante superflue pour les nombres signés.

Question étudiant : Comment l'ordinateur fait-il la différence entre le mot binaire 11111111 qui vaut 255 en non-signé et -1 en complément à deux ?
Réponse du professeur : L'ordinateur ne fait pas de différence intrinsèque sur le motif binaire lui-même ; c'est le type de l'instruction exécutée par le programme (et le mode configuré dans l'ALU) qui indique si le registre doit être interprété en entiers signés ou non signés.

■ Exercice & Démonstration d'Application

Énoncé : Sur une architecture 8 bits, effectuer la soustraction signée : 35 - 75. Détailler l'écriture binaire des deux opérandes en complément à deux, poser l'addition binaire correspondante, donner le résultat décimal et indiquer la valeur du drapeau d'Overflow (OV).

Démarche de résolution :
Étape 1 : Conversion de +35 sur 8 bits. 35 = 32 + 2 + 1 = 00100011_2. Étape 2 : Conversion de -75 en complément à deux sur 8 bits. - Valeur absolue (+75) en binaire naturel sur 8 bits : 64 + 8 + 2 + 1 = 01001011_2. - Inversion de tous les bits (Complément à 1) : 10110100_2. - Ajout de 1 (Complément à 2) : 10110100 + 1 = 10110101_2 (Représentation de -75). Étape 3 : Pose et exécution de l'addition binaire (+35) + (-75). 00100011 (+35) + 10110101 (-75) ---------- 11011000_2 Étape 4 : Interprétation du résultat binaire 11011000_2. Le bit le plus à gauche (MSB) vaut 1, le nombre est négatif. Pour trouver sa valeur absolue en décimal, on réapplique l'algorithme du complément

à deux sur 11011000_2 : - Inversion des bits : 00100111_2 - Ajout de 1 : 00100111 + 1 = 00101000_2 = 32 + 8 = 40 décimal. Le mot binaire 11011000_2 représente donc la valeur -40 en décimal. Étape 5 : Analyse du drapeau de dépassement de capacité (Overflow). On additionne un nombre positif (+35, MSB=0) et un nombre négatif (-75, MSB=1). Les deux opérandes possédant des signes opposés, la production d'un Overflow est impossible.

Résultat : Résultat binaire : 11011000_2 (-40 en décimal). Drapeau d'Overflow OV = 0 (aucun dépassement de capacité). Le calcul 35 - 75 = -40 est exact et valide.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la plage exacte des entiers signés représentables sur un mot mémoire de 16 bits en complément à deux ?

R : De -2^(16-1) à +(2^(16-1) - 1), soit de -32768 à +32767.

Q : Comment détecte-t-on de façon certaine la présence d'un Overflow lors d'une addition de deux entiers signés ?

R : Un Overflow se produit si et seulement si deux opérandes de même signe (deux positifs ou deux négatifs) sont additionnés et produisent un résultat dont le bit de signe (MSB) est opposé.

Q : Quel est le résultat de l'application de la procédure du complément à deux sur la valeur zéro (00000000_2) sur 8 bits ?

R : Inverser tous les bits donne 11111111_2, et ajouter 1 donne 00000000_2 (avec un Carry out de 1 ignoré), ce qui confirme que l'opposé de zéro est zéro.